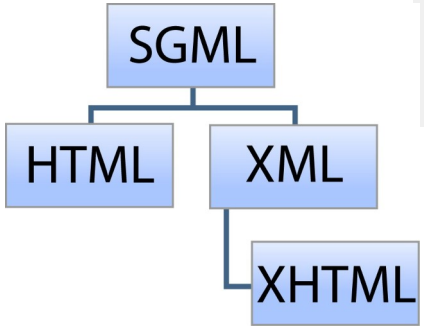
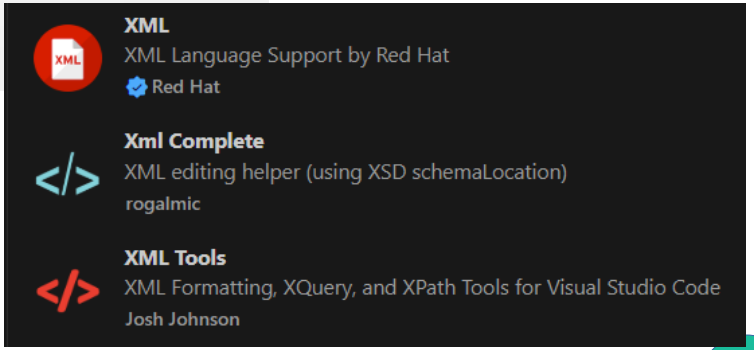
UT4 Lenguajes para el almacenamiento y transmisión de información :XML,JSON

Curso 2024/2025 LENGUAJE DE MARCAS 1º DAW





UT4 Lenguajes para el almacenamiento y transmisión de información XML,JSON

ÍNDICE

1. Introducción a XML
    1.1. ¿Qué es XML?
    1.2. ¿Para qué se usa XML?
    1.3. Diferencias entre XML y HTML
2. Estructura y sintaxis de XML
    2.1. Estructura del Documento XML
    2.2. Elementos en XML
    2.3. Atributos en XML
    2.4. Elementos o Atributos
    2.5. Comentarios
    2.6. Caracteres Especiales
    2.7. Construcción de un documento XML

UT4 Lenguajes para el almacenamiento y transmisión de información XML,JSON

ÍNDICE

3. Validación de XML
    3.1. Validez de un documento XML
    3.2. Formas de Validación
    3.3. Gramáticas
4. DTD (Document Type Definition)
    4.1. ¿Qué es un DTD?
    4.2. Declaración de DTD
        4.2.1 Declaración de Elementos en una DTD
        4.2.2 Declaración de Atributos en una DTD
        4.2.3 Declaración de Entidades en una DTD
5. Como validar documento XML asociado a un DTD

UT4 Lenguajes para el almacenamiento y transmisión de información XML,JSON

ÍNDICE

6. XSD (XML Schema Definition)
    6.1. ¿Qué es un XSD?
    6.2. Diferencias entre DTD y XSD
    6.3. Creación de un XSD
        6.3.1. Estructura de un *schema*
        6.3.2. Cómo vincular un esquema XSD a un documento XML
    6.4. Definición de un elemento
    6.5. Tipos de elementos
        6.5.1. Elementos simples <xs:element>
        6.5.2. Elementos complejos <xs:complexType>
        6.5.3. Elementos simples personalizados <xs:simpleType>
    6.6. Validación de un documento XML con XSD

ÍNDICE

7. XPATH (XML Path Language)
- 7.1. ¿Qué es XPath?
- 7.2. Usos de XPath
- 7.3. ¿Cómo ejecuto una consulta XPath?
- 7.4. El árbol XPath
- 7.5. Tipos de nodos es un árbol XPath
  - 6.5.1. Elementos simples <xs:element>
  - 6.5.2. Elementos complejos <xs:complexType>
  - 6.5.3. Elementos simples personalizados <xs:simpleType>
- 7.6. Sintaxis y notación XPath
- 7.7. Trayecto o ruta de búsqueda
  - 7.7.1. Ejes
  - 7.7.2. Nodos de Comprobación o de búsqueda
  - 7.7.3. Predicados
  - 7.7.4. Operadores aritméticos, lógicos y de comparación.

1. Introducción a XML

1.1. ¿Qué es XML?

XML (Extensible Markup Language) es un lenguaje de marcas **diseñado para almacenar y transportar datos de forma estructurada.**

- No es un lenguaje de programación, sino un formato para estructurar información.
- No define cómo se muestra la información, solo cómo se organiza

```xml
<?xml version="1.0" encoding="UTF-8"?>
<clientes>
    <cliente>
        <nombre>AcerSA</nombre>
        <cif>5664332</cif>
    </cliente>
    <cliente>
        <nombre>Mer SL</nombre>
        <cif>5111444</cif>
    </cliente>
</clientes>
```

1.2. ¿Por qué XML?

✔ Independencia de plataforma (puede usarse en cualquier sistema operativo o lenguaje de programación).
✔ Legibilidad humana y legibilidad para máquinas.
✔ Organización de datos en un formato estructurado.

1.3 Diferencia entre XML y HTML

HTML se usa para **mostrar contenido** en navegadores web (como texto, imágenes y enlaces) usando etiquetas predefinidas, mientras que **XML** se usa para **almacenar y compartir datos** entre sistemas, permitiendo que el usuario cree sus propias etiquetas según sus necesidades.

HTML: Muestra información (estructura visual).

XML: Organiza y transporta datos (no tiene presentación visual).

Característica	XML	HTML
Propósito	Almacenar y transportar datos	Mostrar información en la web
Etiquetas predefinidas	❌ No, el usuario las define	✅ Sí, <h1>, <p>, <table>, etc.
Sensibilidad a mayúsculas	✅ Sí (<nombre> ≠ <Nombre>)	❌ No (<P> = <p>)
Reglas estrictas de estructura	✅ Sí, debe estar bien formado	❌ No, los navegadores pueden corregir errores

1.3 Diferencia entre XML y HTML
Ejemplo de HTML vs XML:

Que XML sea **extensible** significa que puedes crear tus **propias etiquetas** en lugar de usar un conjunto fijo, como en HTML.

Por ejemplo, en HTML debes usar etiquetas como <p> o <h1>, pero en XML **puedes inventar etiquetas según lo que necesites**, como se muestra en el siguiente ejemplo

- **Ejemplo de HTML:**

```
<h1>1984</h1>
<p>Autor: George Orwell</p>
<p>Precio: 12.50€</p>
```

- **Ejemplo de XML:**

```
<libro>
    <titulo>1984</titulo>
    <autor>George Orwell</autor>
    <precio>12.50</precio>
</libro>
```

2.1. Estructura del Documento XML

XML (eXtensible Markup Language) tiene una estructura bien definida y reglas de sintaxis que aseguran que los datos sean organizados correctamente.

Un documento XML tiene siempre esta estructura básica:

Declaración / Versión XML

Nodo raíz

Sub-elementos

...

Cierre del nodo raíz

```
<?xml version="1.0" encoding="UTF-8"? standalone="yes">
<raiz>
    <elemento atributo="valor">Contenido</elemento>
    <elemento>
        <subelemento>Texto</subelemento>
    </elemento>
</raiz>
```

2. Extructura y sintáxis de XML

Declaración XML (Opcional pero recomendada)

La primera línea de un archivo XML debe indicar la versión y la codificación de caracteres

```
<?xml version="1.0" encoding="UTF-8"? standalone="yes">
```

Esta etiqueta tiene los siguientes atributos:

version : Obligatorio a no ser que este ya incluido en otro documento.

encoding : Opcional. Es la codificación de caracteres con la que se ha guardado el documento.

standalone : Opcional. Indica si un documento va acompañado de un DTD o no lo necesita (si no tiene DTD asociado, standalone valdrá yes)

Nodo Raíz

Todos los elementos de un documento XML deben estar contenidos entre las etiquetas de apertura y cierre del nodo raíz, es decir, cuelgan del nodo padre que contiene a todos los demás como si fuera una estructura de árbol.

Sub-elementos hijo

Contendrán todos los elementos con sus datos y atributos.

Final del elemento raíz

Como todas las etiquetas deben tener su etiqueta de cierre, el nodo raíz también.

2.2. Elementos en XML

Los **elementos** son las **etiquetas o nodos**, que organizan la información.
El nombre se coloca entre < >
El texto que aparece entre las etiquetas de apertura y cierre es el contenido

```
<persona>
    <nombre>Juan</nombre>
    <br/>
    <edad>30</edad>
</persona>
```

📌 Reglas importantes:

Etiquetas de cierre:

En XML **es** sintácticamente **ilegal omitir la etiqueta de cierre.** Siempre deben abrirse y cerrarse (<nombre>Juan</nombre>).

En el caso de elementos vacíos, es decir que no tengan contenido, se admite una única etiqueta en lugar del par de etiquetas de apertura/cierre. En esos casos, la etiqueta debe escribirse como <etiqueta/> (poniendo el carácter de etiqueta de cierre después del nombre de la etiqueta). En este caso tenemos

Anidamiento de etiquetas:

Una elemento puede terner a otros elementos como sus hijos, pero **no pueden solaparse**

```
<persona>
    <nombre>Juan</persona>
</nombre>
```

❌

```
<persona>
    <nombre>Juan</nombre>
</persona>
```

✅

2.2. Elementos en XML

📌 Reglas importantes:

Nodo Raíz:

Es la **primera etiqueta** en un documento XML y se llama **etiqueta o nodo raíz**.
Este nodo raíz **es único**.
Todos los demás elementos se anidan en él.
Podemos pensan en el documento de XML como un árbol.

Mayúsculas / minúsculas:

XML es «case-sensitive», es decir que no es lo mismo mayúsculas que minúsculas y que por tanto no es lo mismo <cliente>, que <Cliente> que <CLIENTE>.

Elementos hijo:

Estos deben tener la siguiente estructura:

```
<raiz>
  <hijo>
    <nieto> ... </nieto>
  </hijo>
</raiz>
```

2.3. Atributos en XML

Los **atributos** en XML **sirven para agregar información extra a un elemento** dentro de la etiqueta de apertura. Los atributos en XML son pares nombre = "valor". El valor siempre va entre comillas simples o dobles es indiferente. Se escriben dentro de la etiqueta de apertura en formato:

```
<elemento atributo = "valor"/>
```

📌 Reglas importantes de los atributos en XML

1) Los atributos deben estar dentro de la etiqueta de apertura

❌ Incorrecto:
```
<persona> nombre="Juán"</persona>
```

✅ Correcto:
```
<persona nombre="Juán"/>
```

2) Cada atributo debe tener un valor entre comillas, pueden ser simples o dobles.
3) No pueden repetirse atributos en el mismo elemento
4) Los atributos no pueden contener múltiples valores separados por espacios

❌ Incorrecto:
```
<libro categorias="aventura misterio ciencia"/>
```

✅ Correcto:
```
<libro categorias="aventura" subcategoria="misterio"/>
```

5) Los nombres de los atributos distinguen entre mayúsculas y minúsculas

2.3. Atributos en XML

📌 Reglas importantes de los atributos en XML

6) Se recomienda usar atributos para datos cortos y elementos para datos más estructurados
Usando Atributos

```
<auto marca="Toyota" modelo="Corolla" año="2022"/>
```

Usando Elementos

```
<auto>
    <marca>Toyota</marca>
    <modelo>Corolla</modelo>
    <año>2022</año>
</auto>
```

2.3. Atributos en XML

Ejemplo de XML con atributos y elementos

```
<tienda>
    <libro isbn="001" genero="Ficción">
        <titulo>El Principito</titulo>
        <autor>Antoine de Saint-Exupéry</autor>
        <precio moneda="USD">10.99</precio>
    </libro>

    <libro isbn="002" genero="Ciencia">
        <titulo>Breve historia del tiempo</titulo>
        <autor>Stephen Hawking</autor>
        <precio moneda="EUR">15.50</precio>
    </libro>
</tienda>
```

Explicación del ejemplo:

Atributos:
isbn="001" e isbn="002" identifican cada libro.
genero="Ficción" y genero="Ciencia" indican la categoría.
moneda="USD" y moneda="EUR" especifican la divisa del precio.

Elementos:
<titulo>, <autor> y <precio> contienen la información detallada de cada libro.
<tienda> es el elemento raíz que agrupa todo.

¿Por qué usar ambos?
✔ Los atributos (isbn, genero, moneda) son datos cortos que describen el elemento.
✔ Los elementos (titulo, autor, precio) almacenan datos estructurados.

2.4. ¿Elementos o Atributos?

No existen reglas sobre cuando utilizar atributos o elementos hijos. Sin embargo, como norma general, se debería tender a utilizar los elementos hijos en lugar de los atributos.

Además, el uso de atributos tiene algunos problemas:

- Los atributos no pueden contener generalmente valores múltiples, mientras que los elementos sí.
- Los atributos son difíciles de expandir en el caso de que se deeen hacer cambios futuros en la estructura de los datos.
- Los atributos no permiten estructurar la información.
- Los atributos son más difíciles de manipular por las aplicaciones.
- Los valores de los atributos son difíciles de verificar frente a una DTD.

Sin embargo, hay ocasiones en las que el uso de atributos si puede ser recomendable.

En el ejemplo de la imagen el atributo ID es solamente un contador de mensajes y no una parte de los datos.

En este caso sí podemos decir que el uso de los atributos está recomendado. La información que contiene es los que se denomina metainformación (información sobre la información).

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<mensajes>
  <nota ID="001">
    <para>Elisa</para>
    <de>Pedro</de>
    <titulo>Recordatorio</titulo>
    <cuerpo>No olvides nuestra cita!</cuerpo>
  </nota>
  <nota ID="002">
    <para>Juan</para>
    <de>Francisco</de>
    <titulo>Cita</titulo>
    <cuerpo>Quedamos a comer en el Restaurante de abajo.</cuerpo>
  </nota>
</mensajes>
```

2.5. Comentarios

La sintaxis de los comentarios es la misma (<!-- ... -->) en HTML y XML, pero en XML hay más restricciones.

En HTML se pueden usar para desactivar código temporalmente, mientras que en XML se usan solo para anotaciones.

```
<libro>
    <!-- Este es un comentario sobre el libro -->
    <titulo>XML Básico</titulo>
</libro>
```

2.6. Caracteres Especiales

En XML, algunos caracteres tienen un significado especial y no pueden usarse directamente en el contenido o los atributos porque podrían causar errores en la estructura del documento. Para evitar esto, se utilizan entidades de escape.

Carácter	Significado en XML	Entidad de Escape	Ejemplo de uso
<	Indica el inicio de una etiqueta	<	<mensaje>5 < 10</mensaje>
>	Indica el final de una etiqueta	>	<mensaje>10 > 5</mensaje>
&	Inicia una entidad o referencia	&	<mensaje>Usa & para "y"</mensaje>
"	Se usa en valores de atributos	"	<mensaje texto="Hola "Juan""/>
'	También se usa en valores de atributos	'	<mensaje texto='Es una 'prueba''/>

¿Cuándo se usan estos caracteres especiales?

- Cuando un texto incluye <, > o &, se deben reemplazar por <, > y &.
- En atributos, si el texto contiene comillas dobles (") o simples ('), se deben escapar con " y '.
- No es necesario escapar caracteres como tildes o eñes (á, é, ñ, ü), porque XML soporta Unicode.

2.7. Construcción de un documento XML

En resumen, para crear un documento XML correctamente, es importante seguir una serie de reglas fundamentales que garantizan su correcta estructura y funcionamiento.

- XML es «case-sensitive», es decir que no es lo mismo mayúsculas que minúsculas y que por tanto no es lo mismo <cliente>, que <Cliente> que <CLIENTE>.
- Obligatorio: solo un elemento raíz.
- En general, la costumbre es poner todo en minúsculas.
- Solo se puede poner una etiqueta que empiece por letra o _. Es decir, esta etiqueta no funcionará en los programas <12Cliente>.
- Aparte de eso, una etiqueta sí puede contener números, por lo que esta etiqueta sí es válida <Cliente12>.
- Aunque no es obligatorio a menudo se suele poner en la primera línea un prólogo que indica la versión de XML que estamos usando y la codificación con la que nuestro editor almacena los archivos.

3.1 Validez de un documento XML

Un documento XML puede «estar bien formado» o «ser válido». Se dice que un documento «está bien formado» cuando respeta las reglas XML básicas. Si alguien ha definido las reglas XML para un vocabulario, podremos además decir si el documento es válido o no, lo cual es mejor que simplemente estar bien formado.

Por ejemplo, los siguientes archivos ni siquiera están bien formados.

En este caso la etiqueta <nombre> no está cerrada

```
<clientes>
        <cliente>
                <nombre>AcerSA
                <CIF>5666333</CIF>
        </cliente>
</clientes>
```

En este caso, se ha puesto <cif> cerrado con </CIF> (mayúsculas).

```
<clientes>
        <cliente>
                <nombre>AcerSA</nombre>
                <cif>5666333</CIF>
        </cliente>
</clientes>
```

En este caso,se ha utilizado la admiración, que no es válida (de hecho, el coloreador de sintaxis automático descubre que no es XML y el fichero se muestra de manera literal)

```
<clientes>
        <cliente>
                <nombre!>AcerSA</nombre!>
                <CIF>5666333</CIF>
        </cliente>
</clientes>
```

3.1 Validez de un documento XML

En este caso, el problema es que hay más de un elemento raíz.

```
<cliente>
        <nombre>AcerSA</nombre>
        <CIF>5666333</CIF>
</cliente>
<cliente>
        <nombre>ACME</nombre>
        <CIF>455321</CIF>
</cliente>
```

En general, podemos asumir que un documento puede estar en uno de estos estados que de peor a mejor podríamos indicar así:

1) Mal formado (lo peor)
2) Bien formado.
3) Válido: está bien formado y además nos han dado las reglas para determinar si algo está bien o mal y el documento XML cumple dichas reglas. Este es el mejor caso.

Para determinar si un documento es válido o no, se puede usar:

https://www.freeformatter.com/xml-validator-xsd.html https://www.truugo.com/xml_validator/

3.2 Formas de Validación de un documento XML

Un documento XML puede validarse de dos maneras:

1.- Validación con DTD (Document Type Definition):

- Define la estructura del documento XML.
- Especifica qué etiquetas y atributos se permiten.

Ejemplo DTD:

```
<!DOCTYPE libreria [
   <!ELEMENT libreria (libro+)>
   <!ELEMENT libro (titulo, autor, anio, precio)>
   <!ATTLIST libro id ID #REQUIRED>
   <!ELEMENT titulo (#PCDATA)>
   <!ELEMENT autor (#PCDATA)>
   <!ELEMENT anio (#PCDATA)>
   <!ELEMENT precio (#PCDATA)>
   <!ATTLIST precio moneda CDATA #REQUIRED>
]>
```

2.- Validación con XSD (XML Schema Definition):

- Proporciona una validación más potente y detallada.
- Se usa en aplicaciones más avanzadas.

Ejemplo XSD:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
   <xs:element name="libreria">
      <xs:complexType>
         <xs:sequence>
            <xs:element name="libro" maxOccurs="unbounded">
               <xs:complexType>
                  <xs:sequence>
                     <xs:element name="titulo" type="xs:string"/>
                     <xs:element name="autor" type="xs:string"/>
                     <xs:element name="anio" type="xs:int"/>
                     <xs:element name="precio" type="xs:decimal"/>
                  </xs:sequence>
                  <xs:attribute name="id" type="xs:int" use="required"/>
               </xs:complexType>
            </xs:element>
         </xs:sequence>
      </xs:complexType>
   </xs:element>
</xs:schema>
```

Con estas herramientas y procesos, se puede asegurar que un documento XML esté bien formado y validado correctamente para su uso en diferentes aplicaciones.

3.3. Gramáticas

La gramática en XML se refiere a las reglas y estructuras que definen cómo deben organizarse los elementos dentro de un documento XML. Esto se logra mediante DTD (Document Type Definition) o XSD (XML Schema Definition), los cuales especifican qué etiquetas, atributos y tipos de datos son válidos dentro del documento. Básicamente, estas definiciones actúan como una "gramática" que ayuda a validar que un archivo XML cumpla con una estructura predefinida.

Pensemos en el siguiente problema, en el que dos programadores crean aplicaciones con documentos que se almacenan así:

Programador 1

```
<clientes>
    <cliente>
        <nombre>AcerSA</nombre>
        <cif>455321</cif>
    </cliente>
    <cliente>
        <nombre>ACME</nombre>
        <cif>455321</cif>
    </cliente>
</clientes>
```

Programador 2

```
<clientes>
    <cliente>
        <cif>455321</cif>
        <nombre>AcerSA</nombre>
    </cliente>
    <cliente>
        <cif>455321</cif>
        <nombre>ACME</nombre>
    </cliente>
</clientes>
```

Está claro, que ninguno de los dos puede leer los archivos del otro, sería crítico ponerse de acuerdo en lo que se puede hacer, lo que puede aparecer y en qué orden debe hacerlo. Esto se hará mediante las DTD.

4. Introducción

Un DTD (Document Type Definition) es un **conjunto de reglas sintácticas para definir etiquetas**. Nos indica qué etiquetas se pueden usar en un documento, en qué orden deben aparecer, cuáles pueden aparecer dentro de otras, cuáles tienen atributos, etc. Toda esta información se guarda en un archivo de texto con extensión **.dtd**.

No es un documento XML y por lo tanto no necesita prólogo, salvo en ciertos casos especiales.

Crear una definición del tipo de documento (DTD) es como crear nuestro propio lenguaje de marcado para una aplicación específica. Pueden ser parte del documento XML, pero se suele colocar aparte para poder reutilizarlo.

Puesto que XML es un sistema para definir lenguajes, quien necesite usar XML para intercambio de datos debe definir su propio DTD.

Un problema que presentan los DTDs es que no siguen la sintaxis XML, sino una propia.

4. DTD (Document Type Definition)

4.1. ¿Qué es un DTD?

Un DTD (Document Type Definition) es un conjunto de reglas que define la estructura y validación de un documento XML.

Permite especificar:

- ✔ Qué elementos y atributos pueden existir en el XML.
- ✔ El orden y jerarquía de los elementos.
- ✔ Cuántas veces puede aparecer un elemento.
- ✔ Qué tipo de datos pueden contener los elementos y atributos.

💡 ¿Para qué sirve un DTD?

- ✔ Garantiza que un archivo XML siga una estructura definida.
- ✔ Facilita la interoperabilidad entre sistemas al mantener un formato estándar.
- ✔ Permite validar XML para asegurarse de que cumple las reglas establecidas.

4.2. Declaración de DTD

Una DTD es como un CSS, puede ir en el mismo archivo XML (DTD interno) o puede ir en uno separado (DTD externo). Para poder subirlos al validador, meteremos la DTD junto con el XML.

1.- DTD Interno:

Se coloca dentro de la declaración <!DOCTYPE> en el mismo documento XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE libreria [
    <!ELEMENT libreria (libro+)>
    <!ELEMENT libro (titulo, autor, anio, precio)>
    <!ELEMENT titulo (#PCDATA)>
    <!ELEMENT autor (#PCDATA)>
    <!ELEMENT anio (#PCDATA)>
    <!ELEMENT precio (#PCDATA)>
]>
<libreria>
    <libro>
        <titulo>El Principito</titulo>
        <autor>Antoine de Saint-Exupéry</autor>
        <anio>1943</anio>
        <precio>15.99</precio>
    </libro>
</libreria>
```

2.- DTD Externo:

Se guarda en un archivo separado (libreria.dtd) y se enlaza desde el XML.

Archivo libreria.dtd:

```
<!ELEMENT libreria (libro+)>
<!ELEMENT libro (titulo, autor, anio, precio)>
<!ELEMENT titulo (#PCDATA)>
<!ELEMENT autor (#PCDATA)>
<!ELEMENT anio (#PCDATA)>
<!ELEMENT precio (#PCDATA)>
```

Archivo libreria.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE libreria SYSTEM "libreria.dtd">
<libreria>
    <libro>
        <titulo>El Principito</titulo>
        <autor>Antoine de Saint-Exupéry</autor>
        <anio>1943</anio>
        <precio>15.99</precio>
    </libro>
</libreria>
```

4.2.1 Declaración de Elementos en una DTD

Para declarar un elemento en una DTD se utiliza la siguiente sintaxis:

```
<!ELEMENT nombre_del_elemento tipo_de_contenido>
```

En el tipo de contenido se especifica el contenido permitido en el elemento, pudiendo ser:

1) Texto, **(#PCDATA)**.
2) Otros elementos (hijos).
3) Estar vacío, **EMPTY**.
4) Mixto (texto y otros elementos), **ANY**

4.2.1 Declaración de Elementos en una DTD

1) El contenido de un elemento puede ser texto - **(#PCDATA)**

EJEMPLO: En el siguiente documento XML, el elemento "ciudad" puede contener cualquier texto (cadena de caracteres) **(#PCDATA)**:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ciudad [
   <!ELEMENT ciudad (#PCDATA)>
]>

<ciudad>Roma</ciudad>
```

Escribiendo **#PCDATA** (Parsed Character Data) entre paréntesis "()", se ha indicado que el elemento "ciudad" puede contener una cadena de caracteres analizable.

4.2.1 Declaración de Elementos en una DTD

2) Un elemento puede contener a **otros elementos hijos**

EJEMPLO: En el siguiente ejemplo, el elemento "ciudad" contiene a los elementos hijos "nombre" y "pais":

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ciudad [
   <!ELEMENT ciudad (nombre, pais)>
   <!ELEMENT nombre (#PCDATA)>
   <!ELEMENT pais (#PCDATA)>
]>

<ciudad>
  <nombre>Roma</nombre>
  <pais>Italia</pais>
</ciudad>
```

4.2.1 Declaración de Elementos en una DTD

3) Un elemento puede no contener contenido (estar vacío) - **EMPTY**

EJEMPLO: En la DTD interna del siguiente documento XML, se ha declarado el elemento "mayor_de_edad" como vacío, **EMPTY**. Por tanto, debe escribirse <mayor_de_edad/>:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona (nombre, mayor_de_edad, ciudad)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT mayor_de_edad EMPTY>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Elsa</nombre>
  <mayor_de_edad/>
  <ciudad>Pamplona</ciudad>
</persona>
```

Nota: los elementos vacíos no pueden tener contenido, pero sí pueden tener atributos.

4.2.1 Declaración de Elementos en una DTD

4) Un elemento puede definirse para contener contenido mixto - **ANY**

EJEMPLO: En la DTD interna de los siguientes documentos XML, se ha indicado que el elemento "persona" puede contener texto y otros elementos, es decir, contenido mixto, **ANY** y con esto se permite multiples tipos de documentos XML diferentes que pueden ser válidos.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona ANY>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Elsa</nombre> vive en <ciudad>Pamplona</ciudad>.
</persona>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona ANY>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Elsa</nombre> vive en Pamplona.
</persona>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona ANY>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Elsa</nombre>
</persona>
```

Incluso, si el elemento "persona" estuviese vacío, como en este último ejemplo el documento también sería válido.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona ANY>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona/>
```

4.2.1 Declaración de Elementos en una DTD

4) Un elemento puede definirse para contener contenido mixto - **ANY**

Obsérvese que, por ejemplo, también sería válido el siguiente documento XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona ANY>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Elsa</nombre> vive en Pamplona.
</persona>
```

Elementos vacíos en una DTD - **EMPTY**

Para declarar un elemento vacío en una DTD, hay que indicar que su contenido es **EMPTY**. Un ejemplo de ello podría ser el elemento "br" del HTML, el cual sirve para hacer un salto de línea y no tiene contenido. Por ejemplo, el siguiente documento XML sería válido:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE etiquetas_html [
  <!ELEMENT etiquetas_html (br)>
  <!ELEMENT br EMPTY>
]>

<etiquetas_html>
  <br/>
</etiquetas_html>
```

Un elemento vacío puede tener atributos

EJEMPLO Aunque un elemento se declare vacío, no pudiendo contener texto ni otros elementos, sí puede tener atributos:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE etiquetas_html [
  <!ELEMENT etiquetas_html (br)>
  <!ELEMENT br EMPTY>
  <!ATTLIST br descripcion CDATA #REQUIRED>
]>

<etiquetas_html>
  <br descripcion="Salto de línea"/>
</etiquetas_html>
```

En este ejemplo, para el elemento "br" se ha declarado el atributo descripcion de tipo CDATA (Character DATA), es decir, su valor puede ser una cadena de caracteres. Además, se ha indicado que el atributo es obligatorio escribirlo, #REQUIRED.

Elementos con cualquier contenido en una DTD - **ANY**

Cuando en una DTD se quiere declarar un elemento que pueda contener cualquier contenido (bien sea texto, otros elementos o una mezcla de ambos) esto se puede hacer indicando que su contenido es de tipo ANY.

EJEMPLOS: 1.- En el siguiente documento XML, el elemento "cualquier_contenido" contiene tres elementos "texto"

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ejemplo [
  <!ELEMENT ejemplo (cualquier_contenido)>
  <!ELEMENT cualquier_contenido ANY>
  <!ELEMENT texto (#PCDATA)>
]>

<ejemplo>
  <cualquier_contenido>
    <texto>Texto1</texto>
    <texto>Texto2</texto>
    <texto>Texto3</texto>
  </cualquier_contenido>
</ejemplo>
```

2.- Fíjese que, definiendo la misma DTD, también sería válido el siguiente documento XML donde el elemento "cualquier_contenido" solo contiene texto:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ejemplo [
  <!ELEMENT ejemplo (cualquier_contenido)>
  <!ELEMENT cualquier_contenido ANY>
  <!ELEMENT texto (#PCDATA)>
]>

<ejemplo>
  <cualquier_contenido>Texto1. Texto2.Texto3</cualquier_contenido>
</ejemplo>
```

Elementos con cualquier contenido en una DTD - **ANY**

EJEMPLOS:

3.- Asimismo, el elemento "cualquier_contenido" podría contener una mezcla de texto y uno o más elementos.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ejemplo [
   <!ELEMENT ejemplo (cualquier_contenido)>
   <!ELEMENT cualquier_contenido ANY>
   <!ELEMENT texto (#PCDATA)>
]>

<ejemplo>
 <cualquier_contenido>Texto1<texto>Texto2</texto>Texto3</cualquier_contenido>
</ejemplo>
```

Elementos con cualquier contenido en una DTD - **ANY**

EJEMPLOS:

4.- Por otra parte, si el elemento "cualquier_contenido" estuviese vacío, el documento XML seguiría siendo válido:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ejemplo [
   <!ELEMENT ejemplo (cualquier_contenido)>
   <!ELEMENT cualquier_contenido ANY>
   <!ELEMENT texto (#PCDATA)>
]>

<ejemplo>
   <cualquier_contenido></cualquier_contenido>
</ejemplo>
```

En vez de <cualquier_contenido></cualquier_contenido>, también se puede escribir <cualquier_contenido/>.

Elementos con contenido de tipo texto en una DTD - (#PCDATA)

Para declarar en una DTD un elemento que pueda contener texto analizable, se tiene que indicar que su contenido es (#PCDATA), (Parsed Character Data):

EJEMPLOS: 1.- En el siguiente documento XML, el elemento "texto" contiene caracteres:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ejemplo [
  <!ELEMENT ejemplo (texto)>
  <!ELEMENT texto (#PCDATA)>
]>

<ejemplo>
  <texto>Este elemento solo contiene caracteres.</texto>
</ejemplo>
```

2.- Ahora bien, el elemento "texto" podría estar vacío y el documento XML seguiría siendo válido:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ejemplo [
  <!ELEMENT ejemplo (texto)>
  <!ELEMENT texto (#PCDATA)>
]>

<ejemplo>
  <texto></texto>
</ejemplo>
```

En vez de <texto></texto>, también se puede escribir <texto/>.

Secuencias de elementos en una DTD

En una DTD, un elemento (padre) puede ser declarado para contener a otro u otros elementos (hijos). En la sintaxis, los hijos (también llamados sucesores) tienen que escribirse entre paréntesis "()" y separados por comas ",".

EJEMPLO: Para declarar un elemento (padre) que contenga tres elementos (hijos), se puede escribir:

```
<!ELEMENT padre (hijo1, hijo2, hijo3)>
```

EJEMPLOS: 1.- En el siguiente documento XML, el elemento "persona" contiene a los elementos "nombre", "fecha_de_nacimiento" y "ciudad":

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona (nombre, fecha_de_nacimiento, ciudad)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT fecha_de_nacimiento (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Iker</nombre>
  <fecha_de_nacimiento>26-12-1997</fecha_de_nacimiento>
  <ciudad>Valencia</ciudad>
</persona>
```

Secuencias de elementos en una DTD

EJEMPLOS: 1.- A su vez, los hijos también pueden tener sus propios hijos. Así, el elemento "fecha_de_nacimiento" puede contener, por ejemplo, a los elementos "dia", "mes" y "anio":

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona (nombre, fecha_de_nacimiento, ciudad)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT fecha_de_nacimiento (dia, mes, anio)>
  <!ELEMENT dia (#PCDATA)>
  <!ELEMENT mes (#PCDATA)>
  <!ELEMENT anio (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Iker</nombre>
  <fecha_de_nacimiento>
    <dia>26</dia>
    <mes>12</mes>
    <anio>1997</anio>
  </fecha_de_nacimiento>
  <ciudad>Valencia</ciudad>
</persona>
```

Orden de los hijos de un elemento

En un documento XML, los elementos (hijos) de un elemento (padre), deben escribirse en el mismo orden en el que han sido declarados en la DTD

EJEMPLO: El siguiente documento XML no es válido, porque los elementos sucesores (hijos) del elemento "fecha_de_nacimiento" no se han escrito en el mismo orden que en la DTD.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona (nombre, fecha_de_nacimiento, ciudad)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT fecha_de_nacimiento (dia, mes, anio)>
  <!ELEMENT dia (#PCDATA)>
  <!ELEMENT mes (#PCDATA)>
  <!ELEMENT anio (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Iker</nombre>
  <fecha_de_nacimiento>
    <anio>1997</anio>
    <mes>12</mes>
    <dia>26</dia>
  </fecha_de_nacimiento>
  <ciudad>Valencia</ciudad>
</persona>
```

Cardinalidad de los elementos en una DTD

En una DTD, para definir el **número de veces que pueden aparecer los elementos** de un documento XML, se pueden utilizar los operadores de **cardinalidad** mostrados en la siguiente tabla:

Operadores de cardinalidad en DTD		
Operador	**Cardinalidad**	**Significado**
? (interrogación)	0-1	El elemento es opcional, pudiendo aparecer una sola vez o ninguna.
*** (asterisco)**	0-n	El elemento puede aparecer cero, una o más veces.
+ (signo más)	1-n	El elemento tiene que aparecer, obligatoriamente, una o más veces.

Nota: los elementos declarados en una DTD sobre los que no actúe ningún operador de cardinalidad, tendrán que aparecer obligatoriamente una única vez, en el o los documentos XML a los que se asocie.

Cardinalidad de los elementos en una DTD

Operador de cardinalidad "+" (signo más)

EJEMPLO: En el siguiente documento XML, el elemento "nombre" tiene que aparecer una o más veces. En este caso, aparece tres veces:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE personas [
   <!ELEMENT personas (nombre+)>
   <!ELEMENT nombre (#PCDATA)>
]>

<personas>
   <nombre>Ana</nombre>
   <nombre>Iker</nombre>
   <nombre>Elsa</nombre>
</personas>
```

- Si sobre nombre no actuase el operador (+) el documento no sería válido, ya que, el elemento "personas" solo tendría que contener un elemento "nombre".
- En vez de (nombre+), también se puede escribir (nombre)+.

Cardinalidad de los elementos en una DTD

Operador de cardinalidad "*" (asterísco)

EJEMPLO: En la DTD interna del siguiente documento XML, se ha indicado que el elemento "nombre" tiene que aparecer una única vez. Ahora bien, el elemento "ingrediente" tiene cardinalidad (0-n), por tanto, puede aparecer cero, una o más veces:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE receta_de_cocina [
  <!ELEMENT receta_de_cocina (nombre, ingrediente*)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT ingrediente (#PCDATA)>
]>

<receta_de_cocina>
  <nombre>Tortilla de patatas</nombre>
  <ingrediente>Huevo</ingrediente>
  <ingrediente>Patata</ingrediente>
  <ingrediente>Aceite</ingrediente>
  <ingrediente>Sal</ingrediente>
</receta_de_cocina>
```

Cardinalidad de los elementos en una DTD

Operador de cardinalidad "?" (interrogación)

EJEMPLOS: 1.- En la DTD del siguiente documento XML, la cardinalidad del elemento "mayor_de_edad" es (0-1), siendo opcional su aparición, en este caso aparece

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona (nombre, mayor_de_edad?)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT mayor_de_edad EMPTY>
]>

<persona>
  <nombre>Iker</nombre>
  <mayor_de_edad/>
</persona>
```

2.- Así pues, el siguiente documento también es válido, en el que no aparece el elemento "mayor_de_edad".

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona (nombre, mayor_de_edad?)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT mayor_de_edad EMPTY>
]>

<persona>
  <nombre>Iker</nombre>
</persona>
```

Elementos opcionales en una DTD

En la DTD asociada a un documento XML, se pueden declarar elementos que contengan elementos opcionales. Para ello, se utiliza el operador de elección, representado por una barra vertical (|).

Operador de elección "|" (barra vertical)

EJEMPLO En el siguiente documento XML el elemento "articulo" **puede contener un elemento** "codigo" o un elemento "id"; obligatoriamente uno de ellos, **pero no ambos**:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE articulo [
   <!ELEMENT articulo (codigo | id)>
   <!ELEMENT codigo (#PCDATA)>
   <!ELEMENT id (#PCDATA)>
]>

<articulo>
   <codigo>AF-33</codigo>
</articulo>
```

Elementos opcionales en una DTD

Operador de elección "|" y operador "*"

EJEMPLO: En la DTD del siguiente documento XML se indica que el elemento "articulos" puede contener varios elementos "codigo" e "id":

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE articulos [
  <!ELEMENT articulos (codigo | id)*>
  <!ELEMENT codigo (#PCDATA)>
  <!ELEMENT id (#PCDATA)>
]>

<articulos>
  <codigo>AF-32</codigo>
  <id>3891</id>
  <codigo>AF-50</codigo>
  <codigo>AF-89</codigo>
</articulos>
```

Obsérvese que, con el operador "*", en este ejemplo se ha indicado que el contenido del elemento "articulos" tiene cardinalidad (0-n). Por tanto, el elemento "articulos" puede:

- Estar vacío.
- Contener un elemento "codigo".
- Contener un elemento "id".
- Contener un elemento "codigo" y un elemento "id".
- Contener un elemento "codigo" y varios elementos "id".
- Contener un elemento "id" y varios elementos "codigo".
- Contener varios elementos "codigo" y varios elementos "id".

Nótese también que, dentro del elemento "articulos" pueden aparecer elementos "codigo" e "id" en cualquier orden.

Elementos opcionales en una DTD

Operador de elección "|" en una secuencia de elementos

EJEMPLO: En el siguiente documento XML, pueden aparecer cero o más elementos "articulo" que contengan un elemento "codigo" o un elemento "id", y obligatoriamente un elemento "nombre":

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE articulos [
  <!ELEMENT articulos (articulo)*>
  <!ELEMENT articulo ((codigo | id), nombre)>
  <!ELEMENT codigo (#PCDATA)>
  <!ELEMENT id (#PCDATA)>
  <!ELEMENT nombre (#PCDATA)>
]>

<articulos>
  <articulo>
    <codigo>AF-47</codigo>
    <nombre>Martillo</nombre>
  </articulo>
  <articulo>
    <id>2056</id>
    <nombre>Destornillador</nombre>
  </articulo>
</articulos>
```

Elementos opcionales en una DTD

Operador de elección "|" en una secuencia de elementos en una lista de opciones

EJEMPLO: En la DTD del siguiente documento XML se ha indicado que pueden aparecer cero o más elementos "localidad". En el caso de aparecer, cada uno de ellos contendrá los elementos "pais" y "ciudad", o alternativamente un elemento "codigo_postal":

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE localidades [
  <!ELEMENT localidades (localidad)*>
  <!ELEMENT localidad ((pais, ciudad) | codigo_postal)>
  <!ELEMENT pais (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
  <!ELEMENT codigo_postal (#PCDATA)>
]>

<localidades>
  <localidad>
    <pais>España</pais>
    <ciudad>Valencia</ciudad>
  </localidad>
  <localidad>
    <codigo_postal>31015</codigo_postal>
  </localidad>
</localidades>
```

Elementos opcionales en una DTD

#PCDATA en una lista de opciones permite contenido mixto

En DTD, se pueden declarar **elementos mixtos** que **permiten** la **combinación de contenido de texto y otros subelementos** dentro del mismo elemento **en cualquier orden**. Para declarar un elemento mixto, se utiliza la siguiente sintaxis, importante #PCDATA siempre va en primer lugar:

```
<!ELEMENT elemento (#PCDATA | subelemento)*>
```

En esta declaración, elemento es el nombre del elemento mixto. (#PCDATA | subelemento)* indica que **el elemento puede contener** contenido de **texto (#PCDATA) y/o el subelemento** especificado. El asterisco (*) al final indica que el contenido puede repetirse cero o más veces.

EJEMPLO: Fíjese que, el elemento "articulos" de este documento, puede contener contenido mixto, es decir, texto y otros elementos, en el orden que queramos

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE articulos [
  <!ELEMENT articulos (#PCDATA | codigo | id)*>
  <!ELEMENT codigo (#PCDATA)>
  <!ELEMENT id (#PCDATA)>
]>

<articulos>
  <id>8608</id>
  Teclado
  <codigo>AF-18</codigo>
  <codigo>AF-45</codigo>
  Disquetera
  <id>7552</id>
  <id>4602</id>
</articulos>
```

Elementos opcionales en una DTD

#PCDATA en una lista de opciones permite contenido mixto

EJEMPLO: Véase, en este último ejemplo, que el elemento "provincia" puede aparecer cero o más veces, pudiendo contener contenido mixto:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE localidades [
  <!ELEMENT localidades (provincia*)>
  <!ELEMENT provincia (#PCDATA | ciudad | codigo_postal)*>
  <!ELEMENT ciudad (#PCDATA)>
  <!ELEMENT codigo_postal (#PCDATA)>
]>

<localidades>
  <provincia>
    Navarra
    <ciudad>Estella</ciudad>
    <codigo_postal>31015</codigo_postal>
    <ciudad>Tafalla</ciudad>
  </provincia>
  <provincia>
    Valencia
    <codigo_postal>46520</codigo_postal>
  </provincia>
</localidades>
```

4.2.2 Declaración de Atributos en una DTD

Mediante DTD se pueden declarar los tipos de atributos que pueden tener los elementos en un documento XML. La sintaxis para la declaración de tipos de atributos en DTD es la siguiente:

<!ATTLIST *elemento atributo tipo valorPredeterminado*>

Donde:

- *elemento* es el nombre del elemento al que se le va a declarar el atributo.
- *atributo* es el nombre del atributo que se está declarando.
- *tipo* especifica el tipo de datos del atributo. Puede ser CDATA, enumeración o ID/IDREF.
- *valorPredeterminado* indica el valor por defecto del atributo, si es que tiene uno. Puede ser #REQUIRED, #IMPLIED o #FIXED.

4.2.2 Declaración de Atributos en una DTD

Valor predeterminado: **#REQUIRED**, **#IMPLIED, #FIXED o Valor por defecto**

Además del nombre y el tipo del atributo también se puede especificar cómo debe comportarse el parser ante la presencia o ausencia de cierto atributo en un elemento del documento. Existen cuatro posibles alternativas:

- **#REQUIRED**: El ***atributo*** es de carácter ***obligatorio***.
- **#IMPLIED**: El ***atributo*** es ***opcional***.
- **#FIXED valor** entre comillas dobles (") o simples ('): El ***atributo*** toma siempre el ***valor fijo*** que hemos puesto entre comillas.
- **Valor por defecto**. Siempre va entre comillas dobles (") o simples ('). Si no se define ninguna de las alternativas anteriores el ***atributo*** será por defecto opcional (#implied), es decir ***si no ponemos ningún valor tomará el*** que hemos dado ***por defecto***, ***en caso contrario tomará el nuevo valor*** que le demos .

4.2.2 Declaración de Atributos en una DTD

Valor predeterminado: **#REQUIRED**, **#IMPLIED** o **#FIXED o Valor por defecto**

EJEMPLO:

El DTD está incluido en la parte superior, entre <!DOCTYPE ... [...]>. Define los atributos y las reglas que se aplican a los elementos.

Los elementos <articulo> tienen los siguientes atributos:

- **estado**: Atributo obligatorio **(#REQUIRED)** por lo tanto todos los articulos tiene que tener estado tomando uno de los tres valores
- **descripcion**: Atributo opcional **(#IMPLIED)**.
- **categoria**: Tiene un **valor por defecto** "electrónica". Si no se pone toma ese valor y si se pone otro como "educacion" toma el nuevo valor.
- **condicion**: Tiene un **valor fijo** "nuevo". Es decir todos los elementos <artículo> tendrán como valor del atributo condición siempre "nuevo
- **tipo**: Atributo enumerado con valores posibles físico, digital o servicio.
- **codigo**: Atributo de tipo texto (#PCDATA), que puede contener cualquier valor.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE tienda [
  <!-- Atributo obligatorio: El atributo 'estado' debe estar presente -->
  <!ATTLIST articulo estado (disponible | agotado | oferta) #REQUIRED>
  <!-- Atributo opcional: 'descripcion' puede no aparecer -->
  <!ATTLIST articulo descripcion #IMPLIED>
  <!-- Atributo con Valor por defecto: Si no se define 'categoria', será "electrónica" -->
  <!ATTLIST articulo categoria "electrónica">
  <!-- Atributo con Valor fijo: 'condicion' siempre será "nuevo" -->
  <!ATTLIST articulo condicion #FIXED "nuevo">
  <!-- Atributo Enumerado: 'tipo' puede ser uno de los tres valores: 'físico', 'digital' o 'servicio' -->
  <!ATTLIST articulo tipo (físico | digital | servicio) #IMPLIED>
  <!-- Atributo de tipo texto (PCDATA): 'codigo' puede contener cualquier texto -->
  <!ATTLIST articulo codigo #PCDATA #IMPLIED>
]>

<tienda>
  <!-- Atributo #REQUIRED: 'estado' debe estar presente -->
  <articulo estado="disponible" descripcion="Televisor 4K" categoria="electrónica" condicion="nuevo" tipo="físico" codigo="A123">Televisor</articulo>
  <articulo estado="agotado" descripcion="Laptop Gamer" categoria="electrónica" tipo="digital" codigo="A456">Laptop</articulo>
  <articulo estado="oferta" descripcion="Smartphone 5G" categoria="electrónica" tipo="físico" codigo="A789">Smartphone</articulo>
  <!-- Atributo opcional #IMPLIED: No se especifica 'descripcion', pero no hay error por ser Opcional-->
  <articulo estado="agotado" tipo="físico" codigo="A404">Reloj Inteligente</articulo>
  <!-- Atributo con valor por defecto: 'categoria' es "electrónica" si no se especifica como en este caso -->
  <articulo estado="disponible" descripcion="Mouse Inalámbrico" tipo="físico" codigo="A101">Mouse</articulo>
  <!-- Atributo con valor fijo #FIXED: 'condicion' siempre es "nuevo", y en este caso lo hemos omitido -->
  <articulo estado="agotado" descripcion="Teclado Mecánico" categoria="electrónica" tipo="físico" codigo="A202">Teclado</articulo>
  <!-- Atributo Enumerado: 'tipo' puede ser "físico", "digital" o "servicio" -->
  <articulo estado="disponible" descripcion="Curso de Python" categoria="educación" condicion="nuevo" tipo="digital" codigo="A303">Curso</articulo>
  <!-- Atributo de tipo texto (PCDATA): 'codigo' puede contener cualquier texto -->
  <articulo estado="disponible" descripcion="Audífonos Bluetooth" categoria="electrónica" tipo="físico" codigo="xyz123">Audífonos</articulo>
</tienda>
```

4.2.2 Declaración de Atributos en una DTD

Tipo de datos del atributo: **CDATA**, Enumeración o ID/IDREF

En la especificación DTD se pueden declarar diferentes tipos de atributos para los elementos de un documento XML. Los tipos de atributos más comunes son CDATA, enumeración y ID/IDREF.

Atributo CDATA:

- CDATA permite cualquier tipo de datos de caracteres, incluyendo texto, números, símbolos y caracteres especiales.
- No se aplican restricciones especiales a los valores del atributo.

Sintaxis:

```
<!ATTLIST elemento atributo CDATA #IMPLIED>
```

EJEMPLO:

```
<!DOCTYPE galeria [
 <!ELEMENT galeria (imagen+)>
 <!ELEMENT imagen (#PCDATA)>
   <!ATTLIST imagen descripcion CDATA #IMPLIED>
   <!-- Define un atributo llamado descripcion para el elemento imagen.
   Este atributo es opcional (#IMPLIED) y puede contener cualquier string de texto.-->
]>
<galeria>
   <imagen descripcion="Una hermosa puesta de sol sobre el mar.">puesta_del_sol.jpg</imagen>
   <imagen descripcion="Un retrato en blanco y negro.">retrato_bn.jpg</imagen>
   <imagen>imagen_sin_descripcion.jpg</imagen>
</galeria>
```

4.2.2 Declaración de Atributos en una DTD

Tipo de datos del atributo: CDATA, **Enumeración** o ID/IDREF

Atributo Enumeración:

Sintaxis: <!ATTLIST elemento atributo **(valor1 | valor2 | valor3)** #IMPLIED>

- El atributo puede tener un conjunto específico de valores predefinidos.
- Solo se pueden usar los valores enumerados en el atributo.
- Se declaran los valores posibles dentro de paréntesis y separados por tuberías (|).
- Solo puede tomar uno de los valores
- No se pueden usar operadores de cardinalidad *, + o ? por que estos solo son válidos para la declaración de elementos

EJEMPLO:

```
<!DOCTYPE tienda [
  <!ELEMENT tienda (articulo+)>
  <!ELEMENT articulo (#PCDATA)>
    <!ATTLIST articulo estado (disponible | agotado | oferta) #IMPLIED>
    <!-- Define un atributo llamado estado para el elemento articulo, el cual
    puede tener uno de los tres valores: disponible, agotado, o oferta.
    Este atributo es opcional ya que hemos puesto #IMPLIED. -->
]>
<tienda>
    <articulo estado="disponible">Televisor 4K</articulo>
    <articulo estado="oferta">Laptop Gamer</articulo>
    <articulo estado="agotado">Smartphone XYZ</articulo>
    <articulo>Mouse Inalámbrico</articulo> <!-- Sin estado especificado -->
</tienda>
```

4.2.2 Declaración de Atributos en una DTD

Tipo de datos del atributo: CDATA, Enumeración o **ID/IDREF**

Atributo ID/IDREF:

Sintaxis:

```
<!ATTLIST elemento atributo ID #REQUIRED>
<!ATTLIST elemento2 atributo IDREF #IMPLIED>
```

Los atributos ID y IDREF se utilizan para establecer relaciones únicas entre elementos en un documento XML.

- El atributo **ID** debe ser un i**dentificador único en todo el documento**. Además, el valor de un atributo de tipo ID debe cumplir con las reglas de los NMTOKENs, que son cadenas de caracteres que solo pueden contener letras minúsculas, letras mayúsculas, números, puntos (.), guiones medios (-), guiones bajos (_), o el carácter dos puntos (:). Ten en cuenta también que un valor de ID debe comenzar con una letra.
- El atributo **IDREF** hace referencia al **valor de un atributo ID existente en otro elemento.**

4.2.2 Declaración de Atributos en una DTD

Tipo de datos del atributo: CDATA, Enumeración o **ID/IDREF**

Atributo ID/IDREF:

En este DTD:

1) <!ELEMENT red_social (persona+)>: Define que el documento contiene un elemento red_social que a su vez puede contener uno o más elementos persona.
2) <!ELEMENT persona (nombre)> y <!ELEMENT nombre (#PCDATA)>: Define que cada persona contiene un elemento nombre, el cual es un texto (PCDATA).
3) <!ATTLIST persona id ID #REQUIRED>: Define que cada persona debe tener un atributo id único, que es obligatorio.
4) <!ATTLIST persona amigo_id IDREF #IMPLIED>: Define que cada persona puede tener un atributo opcional amigo_id que hace referencia al id de otra persona en el documento.

El ejemplo de XML muestra cómo se estructura un documento que utiliza este DTD, donde cada persona tiene un ID único y puede referenciar el ID de otra persona como su amigo.

EJEMPLO:

```
<!DOCTYPE red_social [
  <!ELEMENT red_social (persona+)>
  <!ELEMENT persona (nombre)>
  <!ELEMENT nombre (#PCDATA)>
    <!ATTLIST persona id ID #REQUIRED>
    <!ATTLIST persona amigo_id IDREF #IMPLIED>
]>
<red_social>
   <persona id="p1">
      <nombre>Juan Pérez</nombre>
   </persona>
   <persona id="p2" amigo_id="p1">
      <nombre>Maria López</nombre>
   </persona>
   <persona id="p3" amigo_id="p2">
      <nombre>Carlos García</nombre>
   </persona>
</red_social>
```

4.2.2 Declaración de Atributos en una DTD

Declaración de varios atributos en un elemento

EJEMPLO: En la DTD del siguiente documento XML se ha indicado que el elemento "f1" puede tener tres atributos (**pais, fecha_de_nacimiento y equipo**):

- En este ejemplo, el atributo **equipo** es obligatorio escribirlo, #REQUIRED.
- El atributo **pais** es opcional y si no ponemos nada toma como valor por defecto España.
- Mientras que, el atributo **fecha_de_nacimiento** es opcional, #IMPLIED.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE deportistas [
  <!ELEMENT deportistas (futbol | f1 | tenis)*>
  <!ELEMENT futbol (#PCDATA)>
  <!ELEMENT f1 (#PCDATA)>
    <!ATTLIST f1 pais CDATA "España">
    <!ATTLIST f1 fecha_de_nacimiento CDATA #IMPLIED>
    <!ATTLIST f1 equipo CDATA #REQUIRED>
  <!ELEMENT tenis (#PCDATA)>
]>

<deportistas>
  <f1 pais="Alemania" fecha_de_nacimiento="03/07/1987" equipo="Ferrari">Sebastian Vettel</f1>
  <f1 equipo="McLaren">Fernando Alonso</f1>
  <tenis>Rafael Nadal</tenis>
</deportistas>
```

En una DTD, cuando se declara más de un atributo para un elemento, como se ha hecho en este caso, no es necesario escribir varias veces <!ATTLIST, pudiéndose escribir, por ejemplo:

```
<!ATTLIST f1 pais CDATA "España"
        fecha_de_nacimiento CDATA #IMPLIED
        equipo CDATA #REQUIRED>
```

4.2.2 Declaración de Atributos en una DTD

Ejemplos de declaraciones de atributos

EJEMPLO: En la DTD del siguiente documento XML se ha indicado que el elemento "f1" puede tener el atributo "pais":

- Para el elemento "f1", **pais** es un atributo definido de tipo **CDATA** (Character DATA), es decir, su valor será una cadena de caracteres.
- Al no indicarse el país de Fernando Alonso, tendrá como valor por defecto es **"España"**.
- Para Sebastian Vettel, al atributo **pais** se le ha asignado **"Alemania"**, que es un valor distinto al **valor_del_atributo**, que por defecto es **"España"**.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE deportistas [
  <!ELEMENT deportistas (futbol | f1 | tenis)*>
  <!ELEMENT futbol (#PCDATA)>
  <!ELEMENT f1 (#PCDATA)>
  <!ATTLIST f1 pais CDATA "España">
  <!ELEMENT tenis (#PCDATA)>
]>

<deportistas>
  <f1 pais="Alemania">Sebastian Vettel</f1>
  <f1>Fernando Alonso</f1>
  <tenis>Rafael Nadal</tenis>
</deportistas>
```

4.2.2 Declaración de Atributos en una DTD

Ejemplos de declaraciones de atributos

EJEMPLO: En la DTD del siguiente documento XML, se indica que el valor del atributo filmografia de un elemento "director", puede ser una lista de valores de atributos ID. En este caso, una lista de valores escritos en el atributo codpel de los elementos "pelicula" que aparezcan en el documento XML:

- Obsérvese que, los valores de la lista de valores de un atributo IDREFS, se escriben separados por un espacio en blanco.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE cine [
  <!ELEMENT cine (peliculas, directores)>
  <!ELEMENT peliculas (pelicula)*>
    <!ELEMENT pelicula (#PCDATA)>
      <!ATTLIST pelicula codpel ID #REQUIRED>
  <!ELEMENT directores (director)*>
    <!ELEMENT director (#PCDATA)>
      <!ATTLIST director filmografia IDREFS #REQUIRED>
]>

<cine>
  <peliculas>
    <pelicula codpel="P1">Avatar</pelicula>
    <pelicula codpel="P2">Mystic River</pelicula>
    <pelicula codpel="P3">The Terminator</pelicula>
    <pelicula codpel="P4">Titanic</pelicula>
  </peliculas>
  <directores>
    <director filmografia="P2">Clint Eastwood</director>
    <director filmografia="P1 P3 P4">James Cameron</director>
  </directores>
</cine>
```

4.2.3 Declaración de Entidades en una DTD

Las **entidades** en DTD (Document Type Definition) son una forma de **almacenar y reutilizar información en** diferentes **partes del** documento **XML**.. Funcionan como "variables" que pueden contener texto, URLs o incluso partes de código XML.

En una DTD se pueden declarar entidades generales y paramétricas (de parámetro).
Las entidades generales pueden ser:

- Entidades generales internas analizables
- Entidades generales externas analizables
- Entidades generales externas no analizables

Las entidades paramétricas pueden ser:

- Entidades paramétricas internas analizables
- Entidades paramétricas externas analizables

Las entidades generales pueden utilizarse en el cuerpo de un documento XML y en su DTD. Sin embargo, las entidades paramétricas solo pueden utilizarse dentro de la DTD.

4.2.3 Declaración de Entidades en una DTD

Entidades generales internas analizables en DTD

Para declarar una entidad general interna analizable (parsed) en una DTD, se utiliza la siguiente sintaxis:

```
<!ENTITY    nombre_entidad "valor_entidad">
```

Donde:

- *nombre* es el nombre de la entidad
- *"valor"* es el texto o símbolo que representa la entidad.

4.2.3 Declaración de Entidades en una DTD

Entidades generales internas analizables en DTD

EJEMPLO: En la DTD del siguiente documento XML, se han declarado tres entidades (**escritor**, **obra** y **fecha**):
Obsérvese que, para referenciar a las entidades, se ha utilizado la sintaxis: *&nombre_entidad;*

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE textos [
  <!ELEMENT textos (texto)+>
  <!ELEMENT texto (#PCDATA)>

  <!ENTITY escritor "Miguel de Cervantes">
  <!ENTITY obra "El Quijote">
  <!ENTITY fecha "29/09/1947">
]>

<textos>
  <texto>&obra; fue escrito por &escritor;.</texto>
  <texto>&escritor; nació el &fecha;.</texto>
</textos>
```

Si este documento XML se visualizase en un navegador web, se vería algo parecido a:

Este fichero XML no parece tener ninguna información de estilo asociada. Se muestra debajo el árbol del documento.

```
-<textos>
   <texto>El Quijote fue escrito por Miguel de Cervantes.</texto>
   <texto>Miguel de Cervantes nació el 29/09/1947.</texto>
 </textos>
```

4.2.3 Declaración de Entidades en una DTD

Entidades generales externas analizables en DTD

En una DTD se pueden declarar dos tipos de entidades generales externas analizables (parsed): privadas y públicas.
Para las **privadas** se utiliza **SYSTEM**, y para las **públicas** se utiliza **PUBLIC**.
La sintaxis en cada caso es:

```
<!ENTITY nombre_entidad  SYSTEM  "URI">
```

```
<!ENTITY nombre_entidad  PUBLIC  identificador_publico "URI">
```

URI (Uniform Resource Identifier) es la dirección del recurso externo, que puede ser una ruta local o una URL en la web.

4.2.3 Declaración de Entidades en una DTD

Entidades generales externas analizables privadas - SYSTEM

EJEMPLO: En la DTD del siguiente documento XML, se ha declarado la entidad **escritor**:

```
<?xml version="1.0" encoding="UTF-8"
standalone="no"?>
<!DOCTYPE textos [
  <!ELEMENT textos (texto)+>
  <!ELEMENT texto (#PCDATA)>

  <!ENTITY escritor SYSTEM "escritor.txt">
]>

<textos>
  <texto>El Quijote fue escrito por &escritor;.</texto>
</textos>
```

Suponiendo que el archivo "escritor.txt" contenga:

```
Miguel de Cervantes
```

En un navegador web (por ejemplo Edge) se podrá ver:

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
▼<textos>
   <texto>El Quijote fue escrito por Miguel de Cervantes.
   </texto>
   <texto>Miguel de Cervantes nació el 29/09/1947.</texto>
 </textos>
```

5. Como validar documento XML asociado a un DTD

Para validar un documento XML asociado a un DTD, puedes seguir estos pasos:

1)Crea un archivo XML válido que esté asociado a un DTD

2)Utiliza una herramienta o método para validar el documento XML con la DTD asociada:

- **Editor de código y algún complemento** que permita la validación de XML con DTD. Por ejemplo Visual Studio Code y la extensión XML de Red Hat.
- **Herramientas online** como truugo.com/xml_validator/.
- **Programa específico** como XMLSpy, Oxygen XML Editor o Altova XMLSpy.
- **Herramientas en línea de comandos**: Si estás trabajando en un entorno de línea de comandos, puedes utilizar herramientas como xmllint o xmlstarlet para validar el documento XML con la DTD asociada. Estas herramientas suelen ofrecer opciones específicas para realizar la validación y mostrar los resultados.

6. XSD (XML Schema Definition)

6.1 ¿Qué es XSD?

El XSD es un **estándar del World Wide Web Consortium (W3C)** que se utiliza para definir la estructura de los documentos XML, permitiendo su validación. Proporciona un conjunto de reglas que permiten especificar los elementos, atributos, tipos de datos y relaciones entre ellos en un documento XML.

Permite establecer la estructura jerárquica de los elementos, especificar los tipos de datos que pueden contener y definir restricciones como valores mínimos y máximos, formatos de datos, entre otros.

Al utilizar XSD, se puede garantizar la consistencia y la integridad de los datos en los documentos XML, lo que resulta especialmente útil en aplicaciones como intercambio de datos, integración de sistemas y validación de entradas en aplicaciones web.

En la actualidad existen dos formas principales de definir la estructura de los documentos XML: mediante DTD y con XSD. Veamos sus diferencias.

6.2 Diferencias entre DTD y XSD

Los esquemas XSD tienen una forma diferente a los DTD de crear reglas para validar archivos XML. Las diferencias principales son las siguientes:

- Los esquemas XSD admiten tipos de datos con verificación, como enteros, fechas y otros tipos.
- Los esquemas XSD permiten agregar restricciones, como valores mínimos y máximos para un número, o especificar el patrón que debe seguir una cadena válida.

6.3 Creación de un XSD

Para crear un documento XSD sigue los siguientes pasos:

1. **Identifica la estructura:** Determina la estructura que deseas definir en el esquema XSD. Esto incluye los elementos raíz, elementos secundarios, atributos y sus relaciones. Si existe, podemos examinar un archivo XML de ejemplo para comprender su estructura y los elementos , atributos y tipos de datos que contiene.
2. **Crea el archivo XSD:** Utiliza un editor de texto o una herramienta de desarrollo XML para crear un nuevo archivo XSD. Puedes nombrar este archivo según tu preferencia, pero es necesaio usar una extensión «.xsd» a la hora de guardarlo (por ejemplo, «esquema.xsd»).
3. **Define los elementos y tipos de datos:** En el archivo XSD, utiliza elementos del lenguaje XSD para definir los elementos del XML y sus tipos de datos. Puedes usar elementos como <xs:element>, <xs:complexType>, <xs:simpleType>, <xs:sequence>, etc.
4. **Asocia el esquema al XML:** En el XML, agrega los atributos xmlns:xsi y xsi:noNamespaceSchemaLocation al elemento raíz para declarar el espacio de nombres XML del esquema de instancia y especificar la ubicación del archivo XSD. Por ejemplo:

```
1.    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2.    xsi:noNamespaceSchemaLocation="esquema.xsd"
```

6.3 Creación de un XSD

Para crear un documento XSD sigue los siguientes pasos:

5. **Define las restricciones y validaciones:** En el archivo XSD, puedes definir restricciones y validaciones adicionales según sea necesario. Esto incluye restricciones de longitud, valores permitidos, relaciones entre elementos, etc.
6. **Valida el esquema:** Utiliza herramientas de validación XML para verificar que el archivo XSD sea válido y no contenga errores de sintaxis.

Ten en cuenta que la creación de un esquema XSD adecuado requiere un buen conocimiento de la estructura del XML y las necesidades específicas de validación de tus datos. Es importante diseñar el esquema de manera precisa para garantizar que el XML cumpla con las restricciones y reglas definidas.

6.3.1 Estructura de un schema

Para crear un XSD, se utiliza un nodo raíz que contiene todo el esquema y se define mediante el uso del prólogo, que incluye la versión XML. El nodo «schema» contiene todos los elementos y atributos que definen la estructura del documento y las restricciones que se deben cumplir. Por ejemplo, se puede utilizar el siguiente prólogo:

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
3.  ...
4. </xs:schema>
```

Como se puede observar, el nodo raíz se define utilizando el espacio de nombres XML «http://www.w3.org/2001/XMLSchema». Esto se especifica utilizando el atributo «xmlns:xs» (o «xmlns:xsd») en el nodo «schema«. El **prefijo «xs» (o «xsd»)** se utiliza como una abreviatura para referirse al espacio de nombres en el resto del esquema.

Es importante destacar que la URL utilizada en el atributo «xmlns:xs» no implica que se deba estar conectado a Internet para trabajar con XML. Simplemente se utiliza como una referencia a la definición de elementos y atributos que se pueden utilizar en el esquema.

6.3.2 Cómo vincular un esquema XSD a un documento XML

Para vincular un esquema XSD externo en un documento XML, se utiliza un mecanismo que asocia el documento con su esquema correspondiente. Esto es útil para validar que el documento cumple con las reglas y estructuras definidas en el esquema. Veamos cómo se hace:

1. **xmlns:xsi:** Se agrega este atributo al elemento raíz del documento XML. Su propósito es declarar el espacio de nombres XML del esquema de instancia (XSI).
2. **xsi:noNamespaceSchemaLocation:** Este atributo se agrega también al elemento raíz y se utiliza para especificar la ubicación del archivo XSD que contiene las definiciones del esquema. El valor de este atributo es la URL o ruta relativa del archivo XSD.

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <root xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:noNamespaceSchemaLocation="esquema.xsd">
3.      ...
4.  </root>
```

Cuando se añaden estos **atributos** al elemento raíz del documento XML, se está indicando que el documento debe ser validado utilizando el esquema XSD especificado en xsi:noNamespaceSchemaLocation. Veamos un ejemplo:

6.3.2 Cómo vincular un esquema XSD a un documento XML

Veamos un **EJEMPLO**:

Ejemplo de archivo XML (**documento.xml**):

```
<?xml version="1.0" encoding="UTF-8"?>
<libro xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:noNamespaceSchemaLocation="esquema.xsd">
   <titulo>Cien años de soledad</titulo>
   <autor>Gabriel García Márquez</autor>
   <anio>1967</anio>
   <genero>Realismo mágico</genero>
</libro>
```

Ejemplo de archivo XSD (**esquema.xsd**):

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
   <xs:element name="libro">
      <xs:complexType>
         <xs:sequence>
            <xs:element name="titulo" type="xs:string"/>
            <xs:element name="autor" type="xs:string"/>
            <xs:element name="anio" type="xs:integer"/>
            <xs:element name="genero" type="xs:string"/>
         </xs:sequence>
      </xs:complexType>
   </xs:element>
</xs:schema>
```

En este ejemplo:

- El archivo XML (**documento.xml**) contiene información sobre un libro, incluyendo título, autor, año y género. Se utiliza el atributo xmlns:xsi para declarar el espacio de nombres XML del esquema de instancia y xsi:noNamespaceSchemaLocation para especificar la ubicación del archivo XSD.
- El archivo XSD (**esquema.xsd**) define la estructura del elemento **libro**, especificando que debe contener los elementos **titulo**, **autor**, **anio** y **genero**, con los tipos de datos apropiados.

6.4 Definición de un elemento

Para declarar o definir un elemento tenemos dos tipos de estructuras:

a) Estructura vacía

```
<xs:tipoelemento/> <!-- Se puede usar también el prefijo xsd -->
```

b) Estructura llena

```
<xs:tipoelemento> <!-- Se puede usar también el prefijo xsd -->
 ...
</xs:tipoelemento>
```

6.4 Definición de un elemento

Un elemento puede contener atributos cuyos valores siempre van entre comillas. Los tipos de atributos que podemos utilizar en los elementos son los siguientes:

Atributo	Descripción
name	Nombre del elemento
type	Tipo simple predefinido, ya sean los estándares o unos propios
maxOccurs	Número máximo de veces que puede aparecer el elemento (p. ej., [0..unbounded])
minOccurs	Número mínimo de veces que puede aparecer el elemento

*unbouded=ilimitado
Por defecto, si no definimos maxOccurs ni minOccurs, el elemento aparece una sola vez.

6.4 Definición de un elemento

EJEMPLO: Vamos a crear un ejemplo sencillo para ilustrar cómo se utilizan algunos de los atributos anteriores. En este ejemplo, definiremos un elemento <peliculas> que puede contener múltiples elementos <pelicula>.

Ejemplo XSD con maxOccurs:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <!-- Definición del elemento "peliculas" que puede contener
   múltiples elementos "pelicula" -->
  <xs:element name="peliculas">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="pelicula" type="xs:string" maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Ejemplo de XMLque sigue el esquema XSD:

```
<peliculas>
  <pelicula>Matrix</pelicula>
  <pelicula>Inception</pelicula>
  <pelicula>Interstellar</pelicula>
</peliculas>
```

En este ejemplo, hemos utilizado el atributo maxOccurs="unbounded" en el elemento <pelicula>. Esto significa que el elemento <peliculas> puede contener múltiples elementos <pelicula>, y el valor «unbounded» indica que no hay un límite superior en la cantidad de veces que el elemento <pelicula> puede aparecer.

En este documento XML, se cumplen las restricciones definidas en el esquema XSD. La etiqueta <peliculas> contiene múltiples elementos <pelicula>, y no hay un límite específico en la cantidad de películas que pueden aparecer.

6.5 Tipos de elementos

En XSD distinguimos entre tres tipos de elementos: simples, complejos y simples personalizados.

6.5.1 Elementos simples *<xs:element>*

Los **elementos simples** son aquellos que **contienen un solo valor sin subelementos**. Estos elementos se definen utilizando el elemento <xs:element> junto con el atributo *type* para especificar el tipo de datos del elemento.

A continuación se muestra un ejemplo de cómo se define un elemento simple en XSD:

```
<xs:element name="texto" type="xs:string" />
```

Existen varios tipos de datos predefinidos en XSD que se pueden utilizar para definir elementos simples mediante el atributo type, como los que se muestran en la tabla de la derecha

TABLA: Tipos de datos simples en XSD (valores del atributo *type*)

Tipo de Datos	Descripción
xs:string	Representa una cadena de texto.
xs:integer	Representa un número entero.
xs:positiveInteger	Representa un número entero positivo.
xs:negativeInteger	Representa un número entero negativo.
xs:boolean	Representa un valor booleano (true o false).
xs:date	Representa una fecha en formato: YYYY-MM-DD.
xs:time	Representa una hora en formato: HH:MM:SS.
xs:decimal	Representa un número decimal.
xs:float	Representa números de punto flotante de 32 bits de precisión simple.
xs:double	Representa números de punto flotante de 64 bits de doble precisión.

6.5.1 Elementos simples *<xs:element>*

Tipos de declaración de elementos simples (fixed, default)

Si se quiere indicar que un valor es fijo **(fixed)**, se puede escribir, por ejemplo:

```
<xs:element name="mes" type="xs:string" fixed=”agosto”/>
```

También, se puede especificar un valor por defecto **(default)**, por ejemplo, tecleando:

```
<xs:element name="mes" type="xs:string" default=”agosto”/>
```

6.5.1 Elementos simples *<xs:element>*

Veamos algunos**EJEMPLOS DE USO**:

1.- Ejemplo: En este ejemplo, se define dos elementos de tipo simple llamados «nombre» y «edad» donde nombre tiene por tipo de dato simple «xs:string» y edad tiene tipo de dato simple «xs:integer».

```
<xs:element name="nombre" type="xs:string"/>
<xs:element name="edad" type="xs:integer"/>
```

Ejemplo de XML que sigue el esquema XSD: como vemos aparece solo uno de los elementos hijos

```
<nombre>Elsa</nombre>
<edad>23</edad>
```

6.5.2 Elementos complejos <xs:complexType>

El elemento <xs:complexType> se utiliza para definir tipos de datos complejos que tienen sub-elementos y/o atributos. Los tipos de datos complejos permiten especificar la estructura interna de un elemento XML, es decir, sus atributos y elementos hijos.

Dentro del elemento <xs:complexType>, se pueden agregar elementos como los siguientes:

Elemento	Descripción
<xs:sequence>	Se utiliza para especificar una secuencia ordenada de elementos hijos. Los elementos deben aparecer en el orden indicado.
<xs:choice>	Se utiliza para especificar que solo uno de los elementos hijos puede estar presente en un momento dado. Solo uno de los elementos enumerados dentro de **<xs:choice>** puede aparecer en el XML válido.
<xs:all>	Se utiliza para especificar que los elementos hijos pueden estar en cualquier orden. Todos los elementos enumerados dentro de **<xs:all>** pueden aparecer en cualquier orden en el XML válido.
<xs:attribute>	Se utiliza para agregar atributos a un tipo complejo. Se puede especificar el nombre (name) y el tipo de atributo (type).

6.5.2 Elementos complejos <xs:complexType>

Veamos algunos **EJEMPLOS DE USO**:

1. **Ejemplo <xs:sequence>**: En este ejemplo, se define un tipo complejo llamado «Person» que contiene dos elementos hijos: «FirstName» y «LastName». Ambos elementos tienen un tipo de dato simple «xs:string».

```
<xs:complexType name="Person">
  <xs:sequence>
    <xs:element name="FirstName" type="xs:string"/>
    <xs:element name="LastName" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

Ejemplo de XML que sigue el esquema XSD: como vemos aparecen los dos elementos y en el orden que los hemos declarado en el XSD

```
<Person>
  <FirstName>John</FirstName>
  <LastName>Doe</LastName>
</Person>
```

6.5.2 Elementos complejos <xs:complexType>

Veamos algunos **EJEMPLOS DE USO**:

2. **Ejemplo <xs:choice>**: En este ejemplo, se define un tipo complejo «Animal» que permite que solo uno de los elementos hijos («Cat», «Dog» o «Bird») esté presente en un momento dado.

```
<xs:complexType name="Animal">
  <xs:choice>
    <xs:element name="Cat" type="xs:string"/>
    <xs:element name="Dog" type="xs:string"/>
    <xs:element name="Bird" type="xs:string"/>
  </xs:choice>
</xs:complexType>
```

Ejemplo de XML que sigue el esquema XSD: como vemos aparece solo uno de los elementos hijos

```
<Animal>
  <Cat>Miau</Cat>
</Animal>
```

6.5.2 Elementos complejos <xs:complexType>

Veamos algunos **EJEMPLOS DE USO**:

3. **Ejemplo <xs:all>**: En este ejemplo, se define un tipo complejo «Address» que requiere que todos los elementos hijos («Street», «City» y «Country») estén presentes en cualquier orden.

```
<xs:complexType name="Address">
  <xs:all>
    <xs:element name="Street" type="xs:string"/>
    <xs:element name="City" type="xs:string"/>
    <xs:element name="Country" type="xs:string"/>
  </xs:all>
</xs:complexType>
```

Ejemplo de XML que sigue el esquema XSD: como vemos aparacen todos los elementos en cualquier orden

```
<Address>
  <Street>Main Street</Street>
  <City>Cityville</City>
  <Country>Countryland</Country>
</Address>
```

6.5.2 Elementos complejos <xs:complexType>

Veamos algunos **EJEMPLOS DE USO**:

4. **Ejemplo** **<xs:attribute>**: Para definir un atributo se puede emplear la siguiente sintaxis:

```
<xs:attribute name="nombre_del_atributo" type="tipo_de_dato"/>
```

Para el elemento "curso" siguiente, donde aparece el atributo "grupo":

```
<curso grupo="B">2</curso>
```

Sus definiciones pueden ser:

```
<xs:element name="curso" type="xs:integer"/>
  <xs:attribute name="grupo" type="xs:string"/>
```

- Todos los atributos pueden tomar por valor tipos simples.
- Por otra parte, cuando un elemento tiene al menos un atributo –como es el caso del elemento "curso" en este ejemplo– dicho elemento se dice que es complejo.

6.5.2 Elementos complejos <xs:complexType>

Veamos algunos **EJEMPLOS DE USO**:

5. **Ejemplo** **<xs:attribute>**:
Tipos de declaración de atributos **(fixed, default, optional, required)**

- Para indicar que el valor de un atributo es fijo (**fixed)**, es posible escribir, por ejemplo:

```
<xs:attribute name="grupo" type="xs:string"  fixed="B"/>
```

- Para especificar el valor por defecto **(default)** de un atributo, se puede escribir:

```
<xs:attribute name="grupo" type="xs:string"  default="B"/>
```

- Para indicar que un atributo es obligatorio **(required)** escribirlo, se puede teclear:

```
<xs:attribute name="grupo" type="xs:string"  use="require"/>
```

- Por defecto, si no se indica nada, el atributo será opcional (**optional)**.

6.5.2 Elementos complejos <xs:complexType>

Veamos algunos **EJEMPLOS DE USO**:

6. **Ejemplo** **<xs:attribute>**: En este caso, se agrega el atributo «Age» al tipo complejo «Person» con un tipo de dato «xs:integer».

```
<xs:complexType name="Person">
  <xs:sequence>
    <xs:element name="FirstName" type="xs:string"/>
    <xs:element name="LastName" type="xs:string"/>
  </xs:sequence>
  <xs:attribute name="Age" type="xs:integer"/>
</xs:complexType>
```

Ejemplo de XML que sigue el esquema XSD: como vemos se agrega el atributo «Age» al elemento complejo «Person»

```
<Person Age="30">
  <FirstName>John</FirstName>
  <LastName>Doe</LastName>
</Person>
```

6.5.2 Elementos complejos <xs:complexType>

Veamos algunos **EJEMPLOS DE USO**:

7. **Ejemplo** **<xs:attribute>**: En este ejemplo, hemos definido un elemento «libro» con dos elementos hijos («titulo» y «autor») y dos atributos («id» y «fechaPublicacion»).

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <!-- Definición del elemento "libro" con atributos -->
  <xs:element name="libro">
    <xs:complexType>
      <xs:sequence>
        <!-- Definición de elementos dentro de "libro" -->
        <xs:element name="titulo" type="xs:string"/>
        <xs:element name="autor" type="xs:string"/>
      </xs:sequence>
      <!-- Definición de atributos para el elemento "libro" -->
      <xs:attribute name="id" type="xs:positiveInteger"/>
      <xs:attribute name="fechaPublicacion" type="xs:date"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Ejemplo de XML que sigue el esquema XSD: como vemos se agrega el atributo «Age» al elemento complejo «Person»

```
<libro id="123" fechaPublicacion="2022-02-12">
  <titulo>El Gran Libro de Ejemplos</titulo>
  <autor>Jane Doe</autor>
</libro>
```

6.5.3 Elementos simples personalizados <xs:simpleType>

Además de los tipos de datos predefinidos en la tabla anterior, también es posible definir tipos de datos simples personalizados utilizando el elemento **<xs:simpleType>** con el atributo name para establecer un nombre. De esta forma, podemos agregar restricciones adicionales al valor del elemento, como rangos numéricos, patrones de caracteres, valores permitidos, entre otros.

Dentro del elemento **<xs:simpleType>**, se pueden establecer restricciones utilizando otros elementos como los siguientes:

- **<xs:restriction>**: Para poner rangos, patrones, enumerar posibles valores etc.
- **<xs:list>**: Para definir un tipo de lista.
- **<xs:union>**: Para unir varios tipos definidos anteriormente en uno.

6.5.3 Elementos simples personalizados <xs:simpleType>

6.5.3.1 Elemento <xs:restriction>

La estructura básica del elemento **<xs:restriction>** en XSD es la siguiente:

```
<xs:restriction base="tipoDeDatoBase">
  <!-- Restricciones y atributos adicionales aquí -->
</xs:restriction>
```

En esta estructura, el atributo base especifica el tipo de dato base del cual se está realizando la restricción. Dentro del elemento **<xs:restriction>**, se pueden agregar diferentes restricciones y atributos para definir las reglas específicas que se aplican al tipo de dato base.

En la tabla se muestran las restricciones más comunes que se pueden utilizar dentro del elemento <xs:restriction> en XSD:

Restricción	Descripción
<xs:length>	Especifica la longitud exacta del valor permitido.
<xs:minLength>	Especifica la longitud mínima del valor permitido.
<xs:maxLength>	Especifica la longitud máxima del valor permitido.
<xs:minInclusive>	Especifica el valor mínimo permitido, incluyendo ese valor.
<xs:maxInclusive>	Especifica el valor máximo permitido, incluyendo ese valor.
<xs:minExclusive>	Especifica el valor mínimo permitido, excluyendo ese valor.
<xs:maxExclusive>	Especifica el valor máximo permitido, excluyendo ese valor.
<xs:enumeration>	Especifica un conjunto de valores permitidos para seleccionar.
<xs:pattern>	Especifica un patrón utilizando la sintaxis de expresiones regulares.

6.5.3 Elementos simples personalizados <xs:simpleType>

Veamos algunos **EJEMPLOS DE USO**:

1. **Ejemplo** **<xs:maxLength>**: Restricción de un tipo de dato **string** a una longitud máxima de 10 caracteres.

```
<xs:simpleType name="Nombre">
  <xs:restriction base="xs:string">
    <xs:maxLength value="10"/>
  </xs:restriction>
</xs:simpleType>
```

2. **Ejemplo** **<xs:minInclusive> <xs:maxInclusive>**: Restricción de un tipo de dato integer a un rango específico.

```
<xs:simpleType name="Edad">
  <xs:restriction base="xs:integer">
    <xs:minInclusive value="18"/>
    <xs:maxInclusive value="99"/>
  </xs:restriction>
</xs:simpleType>
```

6.5.3 Elementos simples personalizados <xs:simpleType>

Veamos algunos **EJEMPLOS DE USO**:

3. **Ejemplo** **<xs:enumeration>**: Restricción de un tipo de dato integer a un rango específico.

```
<xs:simpleType name="Puntuacion">
  <xs:restriction base="xs:decimal">
    <xs:enumeration value="1.0"/>
    <xs:enumeration value="2.5"/>
    <xs:enumeration value="4.0"/>
    <xs:enumeration value="5.0"/>
  </xs:restriction>
</xs:simpleType>
```

4. **Ejemplo** **<xs:pattern>**: Restringido a ser una cadena de texto de 5 dígitos numéricos.

```
<xs:simpleType name="codigoPostal">
  <xs:restriction base="xs:string">
    <xs:pattern value="[0-9]{5}" />
  </xs:restriction>
</xs:simpleType>
```

6.5.3 Elementos simples personalizados <xs:simpleType>

6.5.3.2 Elemento <xs:list>

El elemento **<xs:list>** en XSD se utiliza para definir una lista de valores de un tipo de dato específico. Permite especificar múltiples valores separados por un espacio en blanco. Ejemplo:

```
<xs:element name="numeros" type="xs:list">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="\d+(\s+\d+)*" />
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

En este ejemplo, se define un elemento llamado «numeros» que contiene una lista de valores. El tipo de dato del elemento se establece como **<xs:list>**. Dentro de **<xs:list>**, se define un **<xs:simpleType>** que especifica las restricciones del tipo de dato. En este caso, se utiliza una restricción de patrón **<xs:pattern>** para asegurarse de que los valores sean números separados por espacios en blanco.

6.5.3 Elementos simples personalizados <xs:simpleType>

6.5.3.3 Elemento <xs:union>

El elemento <xs:union> se utiliza para definir varios tipos de datos en una única opción de tipo de dato.

1.**Ejemplo.** Unión de diferentes tipos de datos: En este ejemplo, la opción memberTypes del elemento **<xs:union>** especifica que el tipo de dato «tipoUnion» **puede ser una fecha o una hora.**

```
<xs:simpleType name="tipoUnion">
  <xs:union memberTypes="xs:date xs:time" />
</xs:simpleType>
```

2. **Ejemplo:** Unión de tipos de datos numéricos: se define un tipo de dato llamado «numericUnion» que puede ser de tipo **<xs:integer>, <xs:decimal>, <xs:float> o <xs:double>**. Esto permite que el valor sea cualquier número entero o decimal, ya sea de precisión simple o doble.

```
<xs:simpleType name="numericUnion">
  <xs:union memberTypes="xs:integer xs:decimal xs:float xs:double" />
</xs:simpleType>
```

6.6 Validación de un documento XML con XSD

La **validación de un documento XML con XSD (XML Schema)** implica asegurarse de que el documento cumple con las reglas y restricciones especificadas en el esquema XSD. Esto garantiza la integridad y consistencia de los datos en el documento XML.

6.6.1 Herramientas para validar documentos XML con XSD

Para validar un documento XML con XSD utiliza una herramienta como las siguientes:

1. **Editor de código y algún complemento** que permita la validación de XML con DTD como, por ejemplo, Visual Studio Code y la extensión XML Tools o XML de Red Hat.
2. **Herramientas online** como *freeformatter.com/xml-validator-xsd.html*.
3. **Programa específico** como XMLSpy, Oxygen XML Editor o Altova XMLSpy.
4. **Herramientas en línea de comandos** como xmllint o xmlstarlet. Estas herramientas suelen ofrecer opciones específicas para realizar la validación y mostrar los resultados.

6.6.2 Validación XSD con herramientas online

Paso 1: Crear el archivo XML
Crea un archivo llamado **datos.xml** y añade el siguiente contenido:

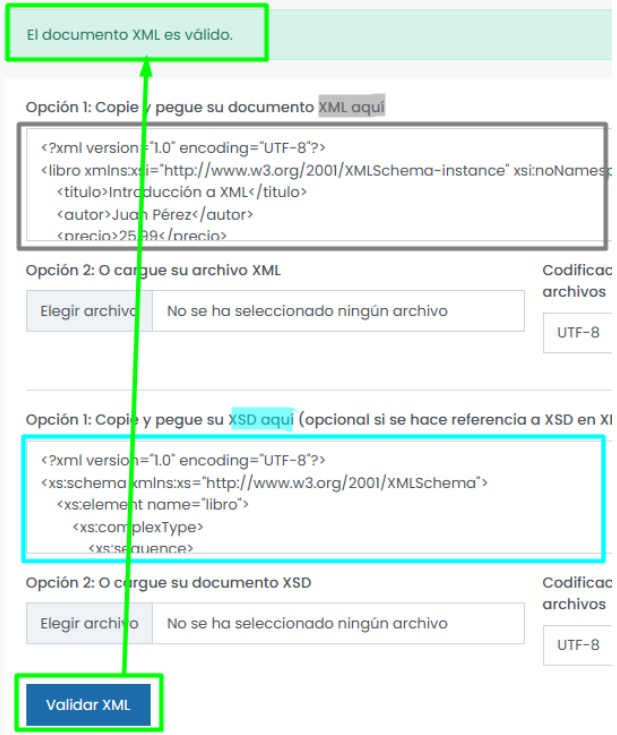
```
<?xml version="1.0" encoding="UTF-8"?>
<libro xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="esquema.xsd">
    <titulo>Introducción a XML</titulo>
    <autor>Juan Pérez</autor>
    <precio>25.99</precio>
</libro>
```

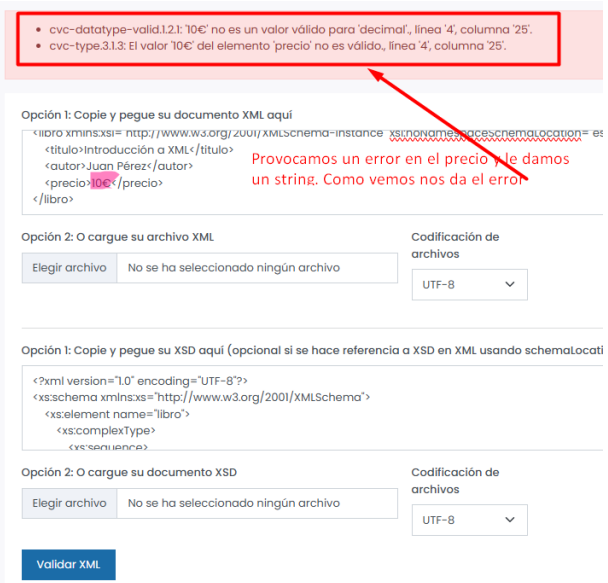
Paso 2: Crear el archivo XSD
Crea un archivo llamado **esquema.xsd** y añade el siguiente contenido:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="libro">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="titulo" type="xs:string"/>
                <xs:element name="autor" type="xs:string"/>
                <xs:element name="precio" type="xs:decimal"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
</xs:schema>
```

6.6.2 Validación XSD con herramientas online

Paso 3: Validar el documento XML con el XSD con herramienta online.
Abre la herramienta online (por ejemplo *freeformatter.com/xml-validator-xsd.html*.) y sube los documentos para verificar la validación.
En la imágen de la izquierda vemos que ha sido validado correctamente el xml con el xsd.
En la imágen de la derecha hemos modificado el valor del precio introduciendo como valor “10€”, es dedir un string en lugar del valor decimal que hemos definido en el xsd, mostrándonos el error al tratar de validarlo.





6.6.3 Validación el documento XML con VS Code y la extensión “XML Tools”

Instalar la extensión XML Tools:

- Abre Visual Studio Code (Ver curso de Visual Studio Code).
- Ve a la sección de extensiones (puedes usar el atajo Ctrl+Shift+X o hacer clic en el icono de extensiones en la barra lateral izquierda).
- Busca «XML Tools» y haz clic en «Install» para instalar la extensión.

Crea tu archivo XML y tu archivo XSD:

Paso 1: Crear el archivo XML
Crea un archivo llamado **datos.xml**

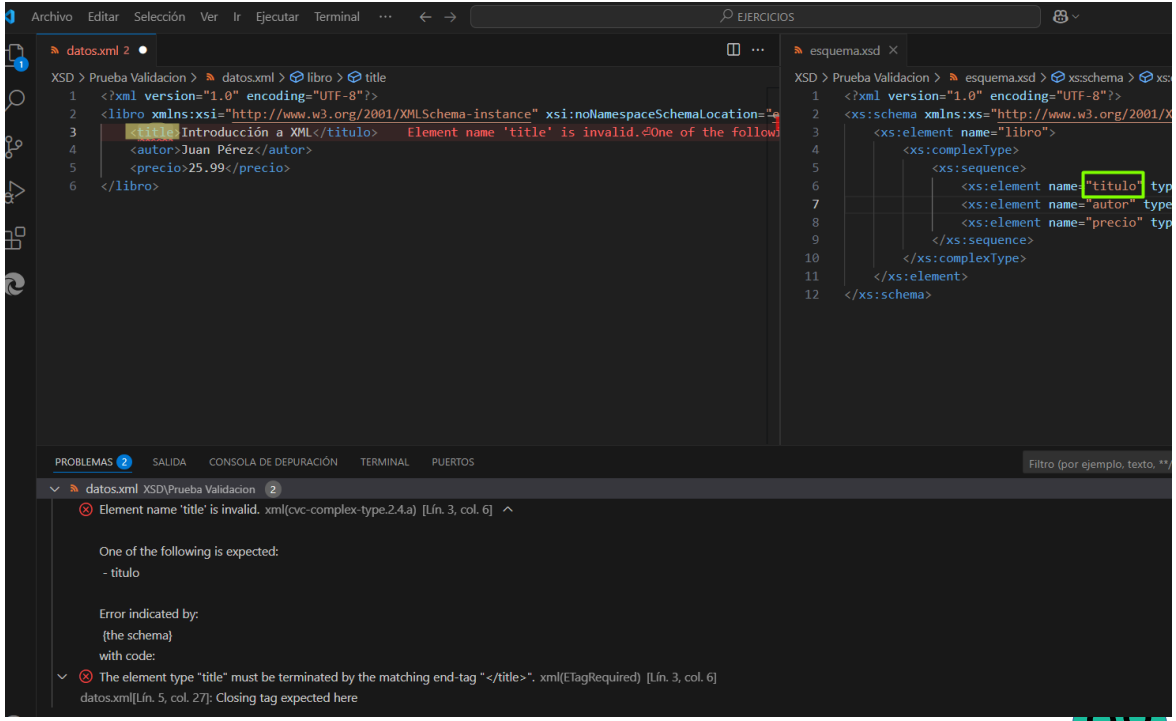
```
<?xml version="1.0" encoding="UTF-8"?>
<libro xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="esquema.xsd">
    <titulo>Introducción a XML</titulo>
    <autor>Juan Pérez</autor>
    <precio>25.99</precio>
</libro>
```

Paso 2: Crear el archivo XSD
Crea un archivo llamado **esquema.xsd**

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="libro">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="titulo" type="xs:string"/>
                <xs:element name="autor" type="xs:string"/>
                <xs:element name="precio" type="xs:decimal"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
</xs:schema>
```

6.6.3 Validación el documento XML con VS Code y la extensión “XML Tools”

Validar el archivo XML: Cuando guardas el archivo XML puedes validar tu archivo XML automáticamente con la extensión «XML Tools»

Visualización de errores:

- Si hay errores en el archivo XML con respecto al esquema (XSD), la extensión «XML Tools» mostrará mensajes de error en la parte inferior de la ventana de Visual Studio Code.
- Puedes hacer clic en los mensajes de error para navegar directamente a la ubicación del problema en el archivo XML.
- Si no ves ningún error, puedes intentar crear uno para ver cómo funciona la extensión. Por ejemplo, en la siguiente imagen se ve que se ha generado un error porque en el esquema XSD se ha definido el elemento «title» pero en el documento XML está declarado como «titulo».